CS5351 Software Engineering
2025/26 Semester A

Technical Debt

Prof. Zhi-An Huang

Email: **huang.za@cityu-dg.edu.cn**

Outline

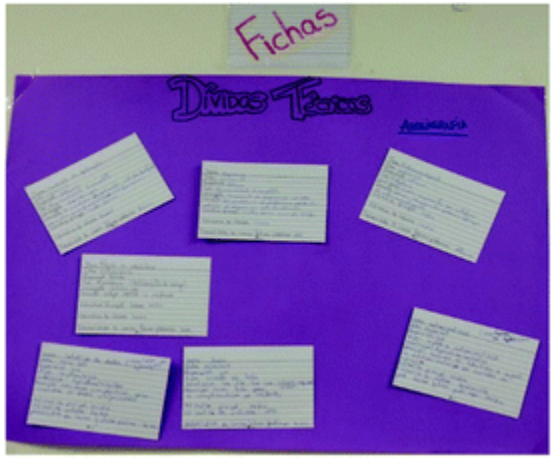
- What is Technical Debt
- Identify Technical Debt
- Manage Technical Debt
- Managing Technical Debts in Practice

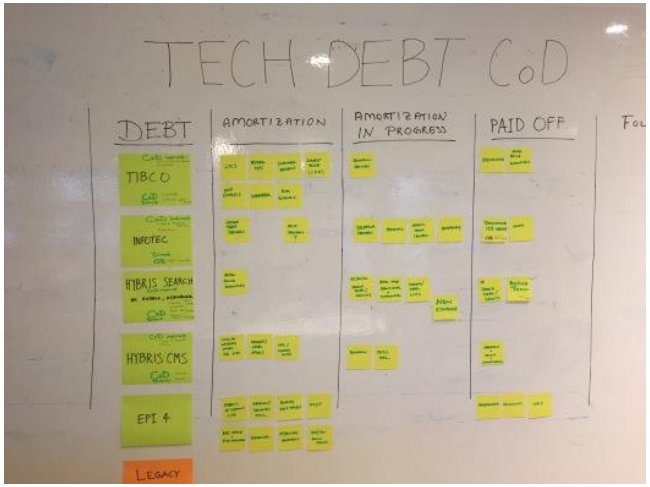
Technical Debt

- Technical debt (TD)
  - is a metaphor originally proposed in 1992 [1]. The metaphor is that **doing things** in a "**quick and dirty**" way **creates a technical debt** (TD).
  - is a design or construction approach that is expedient in the short term but that creates a technical context in which **the same work will cost more to do later than it would cost to do now** (including increased cost over time) [2].
- Example of a decision leading to a TD consequence
  - "We don't have time to reconcile these two databases before our deadline, so we'll write some glue code that keeps them synchronized for now and reconcile them after we ship."

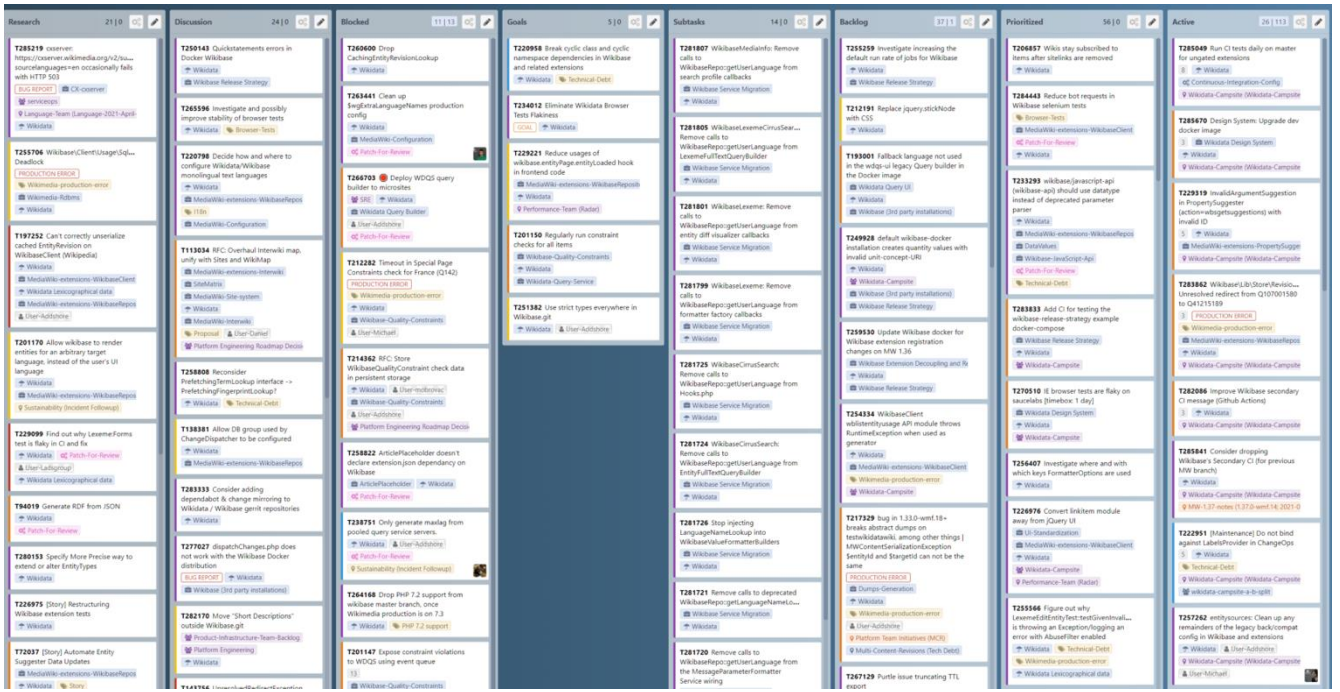
Examples

Technical debt in task board





https://theagilepmo.blog/2016/12/30/technical-debt-visualised-using-cost-of-delay/



https://addshore.com/2021/06/tackling-technical-debt-big-and-small-in-wikidata-and-wikibase/

Effects of Technical Debt in Media

EE Times

HOME NEWS PERSPECTIVES DESIGNLINES PODCAST EDUCATION STORI

DESIGNLINES | EELIFE

Engineers Struggle with Wasted Time and Technical Debt

By Cabe Atwell 09.06.2021 1

Share Post Share on Facebook Share on Twitter

Every workplace has its distractions that can dampen productivity, and the engineering sector is no different, no matter the discipline. Emails, text messages, meetings, and social media also play a significant role in wasted time, which makes it challenging to get to the tasks at hand. Add to that technical debt, and it becomes difficult to stay on track of projects and it generates increased costs. According to a report from Stepsize, engineers spend around six hours per week dealing with technical debt, or the amount of time not working toward critical goals. To put that into another perspective — engineers spend roughly 33% working on maintenance and legacy systems, 50% of which is spent on technical debt. As with money debt, if technical debt isn't paid, it accrues interest, making it tougher to implement changes.

IT Brief AUSTRALIA

How tech debt from legacy solutions will impact business' bottom

By Contributor, 16 Sep 2021

Article by Pure Storage VP & field CTO Mark Jobbins.

Public and private-sector organisations are making massive investments in digital transformation to adapt to the new economy. However, many investments rely on building new applications on legacy IT architecture. This technology approach is leading to rising levels of tech debt, impac organisational performance and increasing cyber-risk.

Organisations that take a half-hearted approach to digital transformation may find themselves with rising levels of tech debt and systems that d business changes. A recent McKinsey survey found that tech debt currently accounts for 40% of IT balance sheets. Sixty per cent of chief inform (CIOs) reported increasing spending on tech debt.

Technical debt occurs because of interdependencies between new systems and legacy IT architecture that often require hard coding. This crea IT infrastructure for the organisation, which is often challenging to maintain. Clear warning signs of tech debt include hard coding and financial a that masquerade as cloud solutions.

Some tech debt is unavoidable because it is a cost of doing business as part of the transition from old to new technology. However, too much te severe financial, operational, and security implications for businesses.

Tech debt increases the total cost of ownership of IT for organisations and directly impacts competitiveness because it is prone to security vulne also unlikely to scale and adapt at the same speed as new technology solutions due to a heavy reliance on manual processes.

Cyber-criminals may find it much easier to target organisations that have higher levels of tech debt due to the likelihood of greater security gaps architecture from disparate systems and a lack of IT integration.

Just like financial debt, organisations cannot afford to sustain tech debt in the long term. It is a critical business issue rather than purely a techno such, tech debt must be addressed in terms of its broader organisational impacts over the longer term.

Consequence of Technical Debt in Media

Forbes

5,132 views | May 30, 2016, 09:33pm

Technical Debt: The Silent Company Killer

Falon Fatemi

Much like a poor diet, technical debt starts with good intentions but ends in frustrating system failures. But unlike unhealthy meals, technical debt is nearly unavoidable: Today's agile developers focus on delivering software quickly to solve a market need. Developers who over-architect or over-code software only delay launch and waste money.

So while shortcuts and assumptions help developers iterate quickly, they also cause software to accrue technical debt. Left unaddressed, this debt bogs down systems, crashes software, and causes multi-month delays in product releases.

Solutions to Technical Debt in Media

Forbes

INNOVATION

Measuring And Managing Technical Debt

Technical debt prevention practices in development

John Kodumal, CTO and cofounder of LaunchDarkly, says, "Technical debt is inevitable in software development, but you can combat it by being proactive: establishing policy, convention, and processes to amortize the cost of reducing debt over time. This is much healthier than stopping other work and trying to dig out from a mountain of debt."

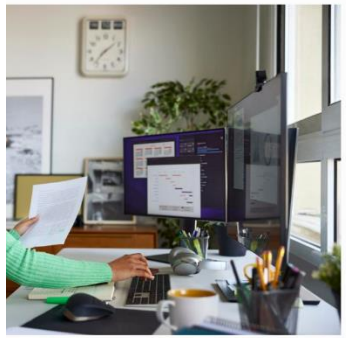
Kodumal recommends several practices, such as "establishing an update policy and cadence for third-party dependencies, using workflows and automation to manage the life cycle of feature flags, and establishing service-level objectives."

To help reduce the likelihood of introducing new technical debt, consider the following best practices:

- Standardize naming conventions, documentation requirements, and reference diagrams.
- Instrument CI/CD and test automation to increase release frequency and improve quality.
- Develop code refactoring as skills and patterns practiced by more developers.
- Define the architecture so developers know where to code data, logic, and experiences.
- Conduct regular code reviews to help developers improve coding practices.

orbes Councils Member
ogy Council COUNCIL POST | Membership (Fee-Based)

ly the CIO at Progrexion. His expertise is in big
nd enterprise security.



f their time dealing with technical debt. Multiple
rage organization wastes 23%-42% of their
chnical debt. In those same studies, CIOs

Forbes

FORBES > INNOVATION > ENTERPRISE TECH

Here's What You Can Do To Manage Your Technical Debt

Forrester Contributor

Follow

0

Sep 6, 2023, 09:00am EDT

- **Demonstrate value.** Those outside of the IT environment may struggle to see the business value of maintaining a technology inventory. There is a cost to managing it, both in the form of people and resources.
- **Ensure process compliance.** Teams, seeing this process as a potential delay or blocker to what they want to do, may find ways to evade TLM when the process doesn't produce the result they want, and over time, this dissatisfaction may result in the TLM process being shut down.
- **Avoid delays.** There is great business cost in a TLM process that takes months; new technologies may be essential for an organization's survival, and requests for new technology should be evaluated in a timely manner.

Maintain an inventory of vendors
es are nonstandard, and
egree of vulnerability.

ure TLM requires automation
arge and distributed
spreadsheets, Forrester
on management database.

InfoWorld UNITED STATES SOFTWARE DEVELOPMENT CLOUD COMPUTING MACHINE LEARNING ANALYTICS IDG TECH(TALK) COMMUNITY NEWSLETTERS

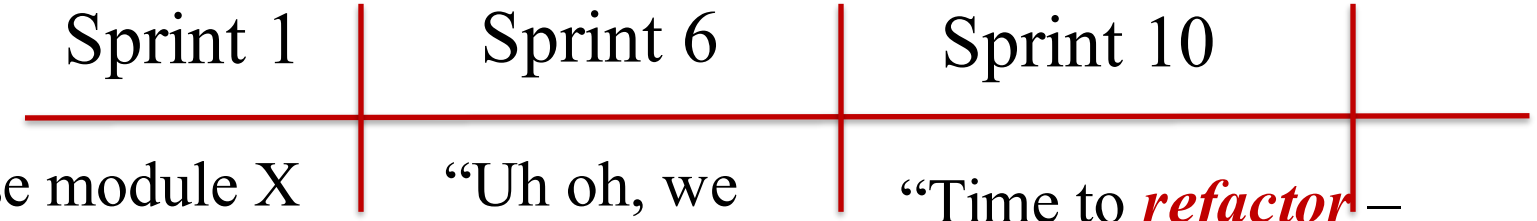
Home > Software Development

How to minimize new technical debt

With best practices and a commitment to not let technical debt grow, developers can make a solid business case, especially when staffing and money are tight.

Technical Debt
incur, then pay interest, finally pay back

- Typical scenario:



"Expose module X as API for other modules to use? We aren't going to need it! ***No need to design*** for it."

"Uh oh, we ARE going to need it! Let's ***work around*** it to make the current release possible first."

"Time to ***refactor*** – we've gotta fix it (the workaround) !"

Note: the API development is a feature, rather than a TD.

Wrong - Incur the technical debt

Pay interest on the technical debt

Pay back the technical debt

Examples of "Interest Payments" on Technical Debt

- *e.g.,* Lack of unit tests on legacy code causes new development, testing, and debugging to take longer
- *e.g.,* Overly simple design does not readily support changes in the environment, and seemingly simple environment changes require massive code rework
- *e.g.,* High product support cost for a buggy system (non-software interest payment)
- *e.g.,* Brittle system means each bug-fix introduces unintended side effects (e.g., new bugs), so each bug-fix becomes a multibug-fix project
- *e.g.,* Bug reports are so frequent that time spent fixing bugs in the production system prevents any work on new functionality
- *e.g.,* Overly lengthy edit-compile-debug times due to poor development environment (non-code cause of technical debt)

Counterexamples of Technical Debts

- Many kinds of delayed or incomplete work are not technical debt:
  - feature backlog, deferred features, cut features

- In general, if "the same (development) work ***will not*** cost more to do later than it would cost to do now", then it is not a technical debt.

Technical Debts in Practice [8]

- Ernst et al. surveyed 1831 software engineers and architects found:
  - 31% stated TD as an implicit part of their backlog
  - 25% stated TD as an explicit part of their backlog
- TD items commonly are not present in the official sprint backlog; they are often documented in quite ad-hoc managed *shadow backlogs* (i.e., “unofficial”, e.g., private notes of the team or a member, comments in source code commits).
- A sprint backlog includes a mix of different items (e.g., new features, bugs/defects, and improvements). TD is often grouped as Improvements.
- When TD is present in the software, **the only significantly effective way of reducing TD is to refactor the software**.

Impacts of Technical Debts

- In some large organizations, development time dedicated to managing Technical Debt is substantial (an average of 25% of the overall development. Some said 40%-50%)

Company View

- Many companies tend to *trade*
  - their longer-term ability to produce new releases frequently, with high quality, quick feedback and small effort

in exchange for

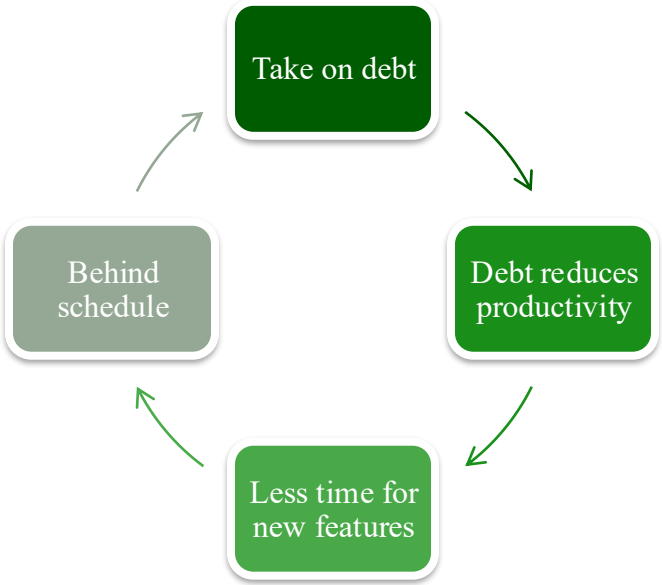
  - short-term features that might quickly fix them but slow them down in the long run.

- This strategy makes these companies vulnerable to their competitors

Two Sides of the Same Story with a Company [2]

- Business View
  - Shorten time to market
  - Preserve startup investment
  - Delay development expense
- Business staff tend to be ***optimistic*** about the benefit and the costs. These attitudes are often ***unconscious***.

- Technical View



- Technical staff tends to be ***pessimistic*** about the benefit and the costs. These attitudes are often ***unconscious***.

Technical Debt and Its Basic Management

- Technical debts have been classified by dimension:
  1. Type (e.g., design, testing, or documentation debt),
  2. Intentionality (intentionally or unintentionally),
  3. Time horizon (short or long term), and
  4. Degree of focus.

Use them to judge a technical debt

- (Popular) Technical Debts management idea
  - **Identify** a technical debt and **track** it through product backlog of the software project. **Discuss** them.
  - E.g., put the following into the backlog: “We don't have time to reconcile these two databases before our deadline, so we'll write some glue code that keeps them synchronized for now and reconcile them after we ship.”

Dimensions of Technical Debts

Dimension 1:
Types of Technical Debt

- Technical Debt is not just a coding problem.
- Refactoring out the code smells **cannot** solve them all.

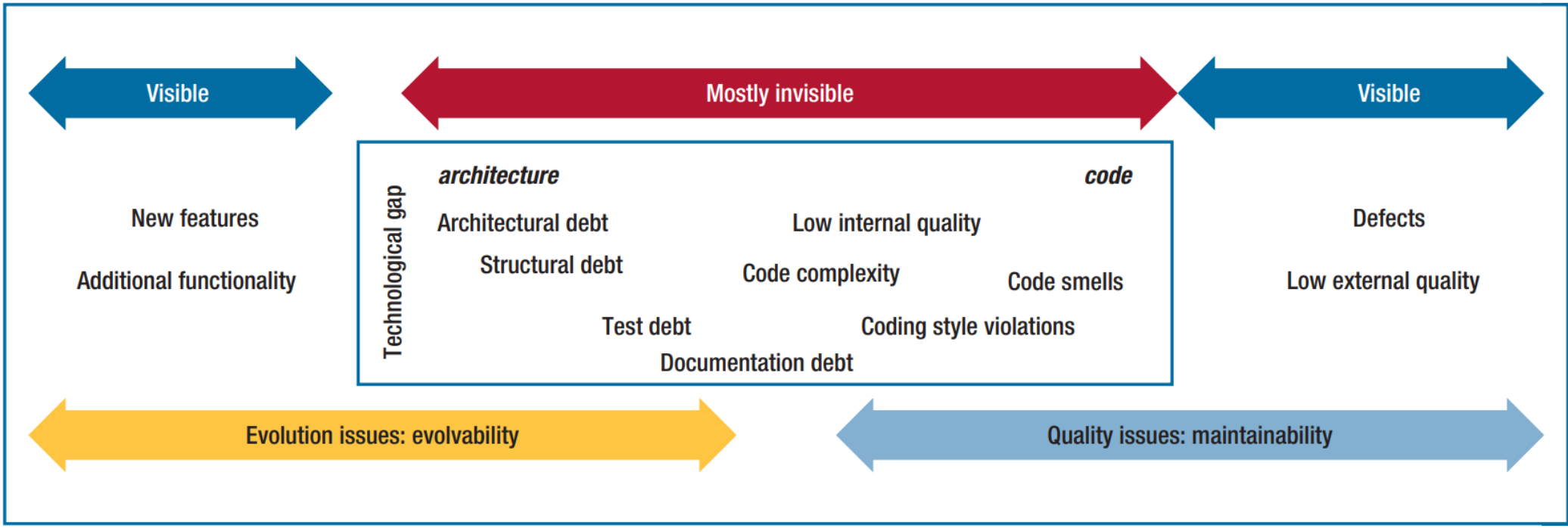


FIGURE 1. The technical debt landscape. On the left, evolution or its challenges; on the right, quality issues, both internal and external. [3]

Dimension 2:
Intentionality of Technical Debt

- Intentional Technical Debt
  - Debt incurred by intention (we intend to commit to do it)
  - e.g., “If we don’t get this release done, there won’t be a second release. Let’s focus on coding with minimal testing effort. No documentation. Operate the system with users in the first few days after the release rather than training them upfront. ”
  - e.g., “We don’t have time to build general support for multiple platforms. We’ll support iOS now and build in support for Android, etc., later. Code only needs to function well in iOS apps.”
  - e.g., “We didn’t have time to write unit tests for all the code we wrote in the last 2 months of the project even though I know we are using a test-driven development approach. We’ll write those after we release the software.”

Dimension 2:
Intentionality of Technical Debt

- Unintentional Technical Debt
  - Debt incurred unintentionally due to low-quality work.
  - e.g., A junior programmer writes bad code
  - e.g., A newcomer/contractor writes code that doesn't follow the coding standard
  - e.g., A major design strategy (e.g., decide to use the model-view-controller architecture to connect all major modules in the software) turns out poorly
  - e.g., A comprehensive refactoring task goes sideways
  - e.g., Honest Mistakes. "I wish we'd known Framework 2.1 would be so much better than Framework 2.0."
  - e.g., Careless Mistakes. "Design? What design? Hardware design?

Dimension 3:
Time Horizon

- Short-Term Debt
  - Usually incurred reactively and actions are done in a short time
  - e.g., Skipping some integration tests to get a release out the door
  - e.g., "We don't have time to implement this the right way; just hack it in, and we'll fix it after we ship."
  - e.g., Violating the coding standards to deploy a critical patch urgently
- Long-Term Debt
  - Usually incurred proactively, for strategic reasons
  - e.g., “We don’t think we’re going to need to support a second platform for at least 3 years, so our design supports only one platform.”
  - e.g., The core part is in COBOL for 40 years. This module should continue to use COBOL as the programming language.

Dimension 4:
Degree of Focus

- Focused Short-Term Debt
  - Individually identifiable shortcuts (e.g., partial implementation of a class)
- Unfocused Short-Term Debt
  - Numerous tiny shortcuts (e.g., not following the coding standard frequently or loose documentation)
- Focused Long-Term Debt
  - e.g., “We don’t think we’re going to need to support a second platform for at least 3 years, so our design supports only one platform.”
- Unfocused Long-Term Debt
  - e.g., Let’s use the most productive languages and frameworks to implement individual features.

Identify Technical Debts

Patterns to Increase Technical Debts

- There are some well-known patterns in software development to *increase* technical debts

1. Schedule Pressure
2. Duplication of Code
3. Get it “right” the first time

Patterns to Increase Technical Debts:

1. Schedule Pressure

- When a team is held to a commitment that is unreasonable, they are bound to cut corners to meet management expectations.
  - E.g., creeping scope of new features, change in team composition, late integration
- Solution
  - Use a more flexible planning approach

Patterns to Increase Technical Debts:
2. Duplication of Code

- Many reasons for code duplications
  - Lack of experience on the part of the team members
  - Copy-and-paste programming practice
  - Conform to the poor design of existing software
  - Pressure to deliver; guess a schedule
- Solution
  - Use static analysis tool for assistance (see Code for Quality slides)
  - Pair programming
    - To spread the knowledge and improve the competence of team members
  - *Either* (a) Repay debts now *or* (b) Track it if replay later,
    - (1) fix it now, (2) add a runtime exception to the location of technical debts and fix it a few minutes later, (3) add the debt (location, description, potential cost of not fixing it) to the project backlog

Patterns to Increase Technical Debts:

3. Get it "right" the first time

- Opposite of duplication – create a lot of powerful design up-front.
  - We have over-engineered and over-generalized our intended product.
- Costs associated with fixing *keep growing*

Manage Technical Debts

Mange Technical Debts [5]

- If we can't avoid technical debt, we must manage it.
- This means recognizing it, tracking it, making reasoned decisions about it, and preventing its worst consequences.

Methods to Manage Technical Debts:

Tracking Technical Debts [2]

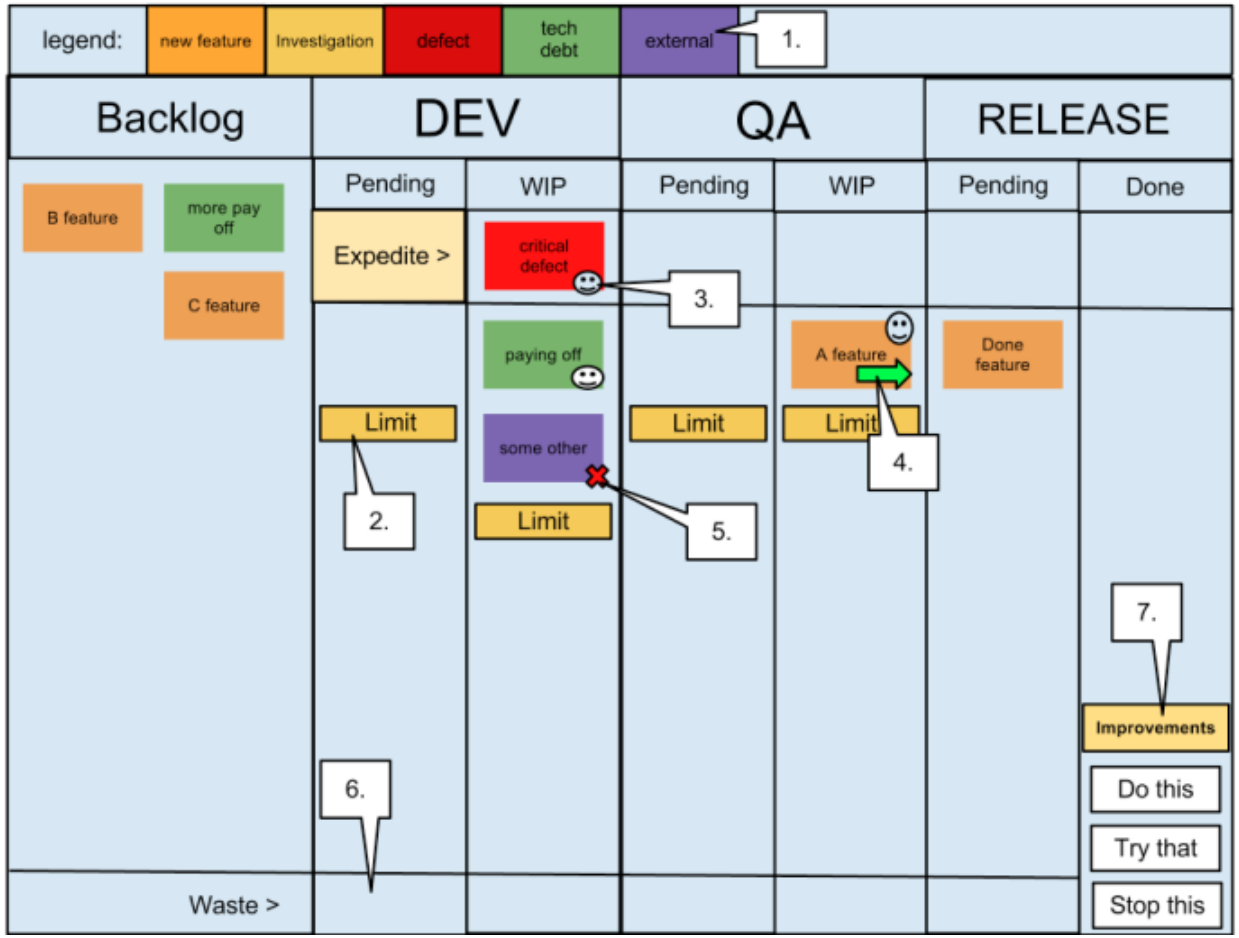
- Tracking technical debts (TD) is necessary to manage them.
  - All "good debt" can be tracked (at least by definition)
  - Log as defects
  - Include in product backlogs
  - Monitor project velocity
  - Monitor the amount of rework
- Ways to Measure Debt
  - Total debt in the product backlog
  - Maintenance budget (or fraction of maintenance budget)
  - Measure debt in money, not features, e.g., "50% of the R&D budget is nonproductive maintenance work"

Methods to Manage Technical Debts:
Tracking Technical Debts Visually [6]

- Visualize the technical debts on task boards. The lower section includes three refactoring tasks (i.e., instances of technical debts) to be addressed in a project: one is under code review, another one is "in progress", and the last one has not been handled at all.

Methods to Manage Technical Debts:
Tracking Technical Debts Visually [6]

- Visualize the technical debts on Kanban board. (for the 1-7, see the next slide)



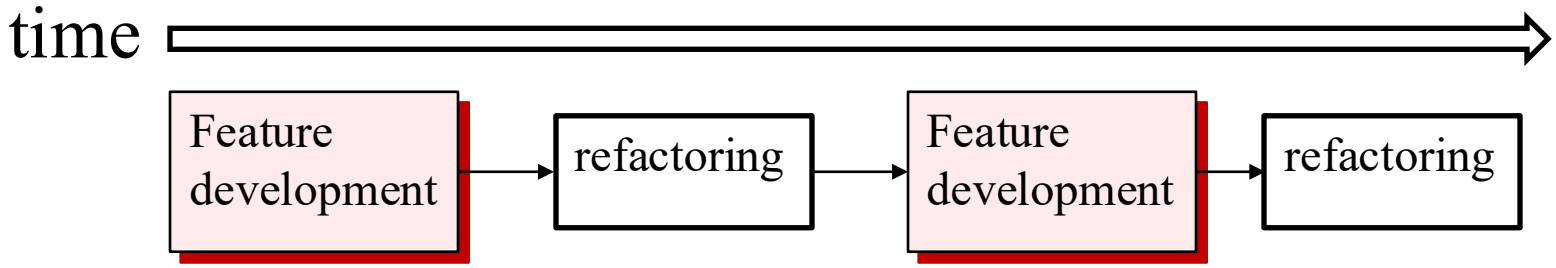
The seven legends on the last slide

- **Legend** – This shows the types of cards (by color) that can be on the board, these include New features, Tech debt, Defects, Investigations (support) and External Issues (e.g. new servers). The purpose of this is to clearly **visualize** the type of work that is being processed
- **Limits** – Each pending and work in progress column has a limit to control the amount of work that can be taken on. These limits are set based on resource levels and other **bottlenecks**, and are intended to encourage the PULLING of work when capacity exists rather than continuing to force more work onto a bottleneck.
- **Avatar** – Each team member has ONE avatar to place on the item they are on, this tells other team members that this card is being worked upon and should be progressing. As we pair on most development, there will often be more than one avatar on each Dev task.
- **Arrows** – Indicate movement of tasks since the last standup, again this is part of the **visualization** of the process. These cards are often ignored at the standup, as we want to focus on the items that are not progressing, such as…
- **Blockers** – We want to minimize blockers as they cause more bottlenecks in the system, so we clearly indicate these items with blocked avatar. An additional requirement to blocking a card is to add the details of WHO and WHY the item is unable to proceed.
- **Waste** – Tasks can be abandoned at any stage in the process, we collect these and review at the weekly **retrospective**.
- **Continuous Improvements** – Also, at the weekly **retrospective** we decide upon three improvement points that we want to tackle for the following week. These are summarized on the board and recalled at the beginning of the morning standup, if we fail to make any progress on an improvement then it may be rolled over to the next week.

Methods to Manage Technical Debts:

Repay Technical Debts

- An effective tactic is to periodically reserve a timeslot to deal with technical debt.



- Longer time for refactoring to trade for a smaller number of refactoring cycles, and vice versa.
  - Prioritize refactoring tasks to address the critical areas first.
  - Involve product owners and stakeholders to get their perspectives

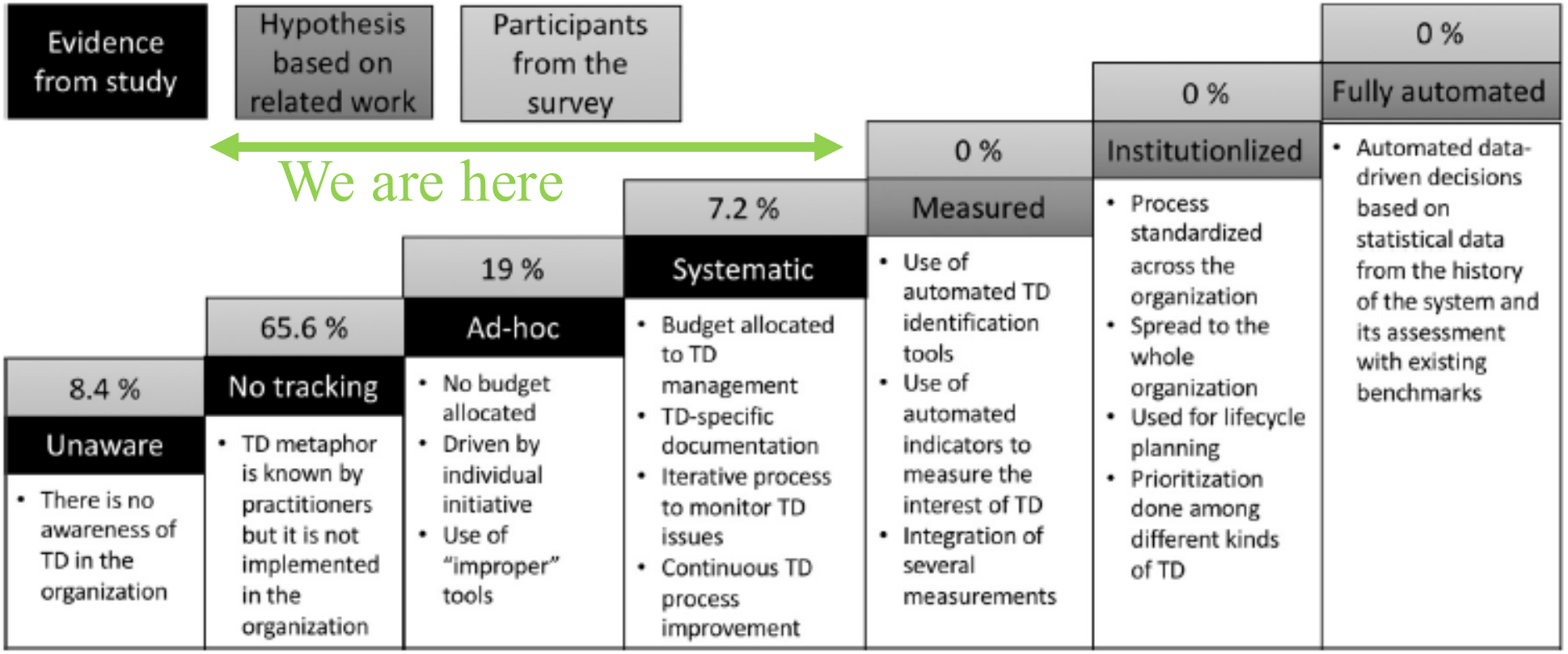
Methods to Manage Technical Debts:
Discuss Technical Debts

- Educate technical staff about business decision-making involved in the project
  - Fit developers into the business context
- Educate business staff about technical decision-making
  - Also serve as an aspect of the inputs to their business or operational decisions
- Raises awareness/transparency of important issues that are often covered up
  - As a risk aversion measure
- Allows TDs to be managed more explicitly and visually
  - As a starting point for a project to resolve them individually.

Managing Technical Debts in Practice

- Although developers may have a desire for better ways of doing all these things, and yet they do not do it.
- Strategies in workplaces (from a survey [5] on 26 companies)
  1. Do nothing—"if it ain't broke, don't fix it"—because the debt might never become visible to the customer.
  2. Use a risk management approach to evaluating and prioritizing technical debt's cost and value. E.g., allocate 5 to 10 percent of each release cycle to addressing technical debt.
  3. Manage the expectations of customers and nontechnical stakeholders by making them equal partners and facilitating open dialogue about the debt's implications.
  4. Conduct audits with the development team to make technical debt visible and explicit; track it using a backlog/task board.

Managing Technical Debts in Practice [7]



Context of the survey [7] : 226 participants from 15 large organizations with different roles (developers, architects, managers)

Managing Technical Debts in Practice [7]

highest

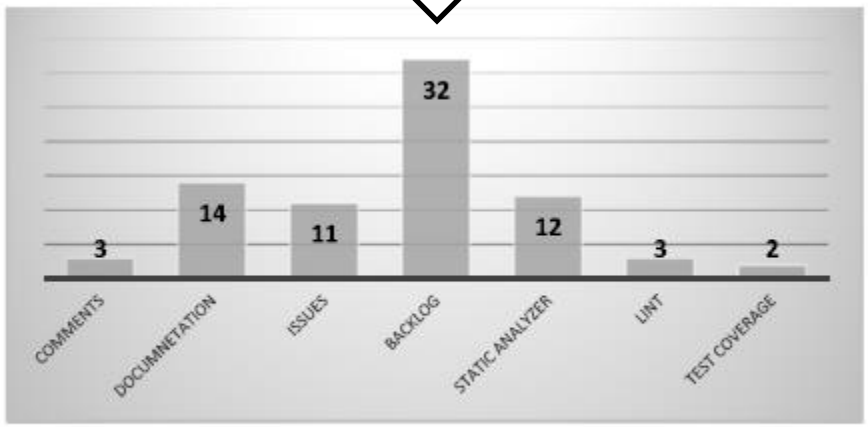


Fig. 13. Number of participants using a specific kind of tool.

lowest

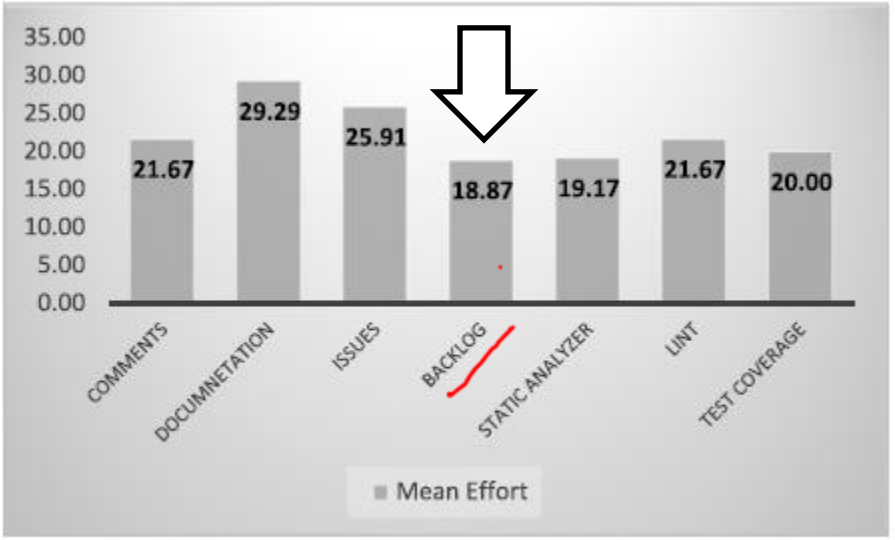


Fig. 15. Mean of management effort for each kind of tool.

Managing Technical Debts in Practice [7]

- **[Effective] Backlogs, static analyzers and "lint" programs** all **increase the tracking level**, but we cannot see a big difference (although static code analyzers seem to contribute better to the participants' awareness). They are also the ones with the **least overhead**. They therefore seem to be considered the **best practices** at the moment **to track TD**.
  - **[Effective] Backlogs are the most used tool** among the participants. In particular, the most used backlog tools are Jira, Hansoft, and Excel.
- **[Ineffective] Comments** in the code help awareness, but they are **not considered tracking**, and they are used by just 1% of the respondents. This is probably because they are not used in a document that can be monitored by the team outside the code.
- **[Ineffective] Documentation** increases TD awareness, but it is **not considered as a high level of tracking**, and it has the **highest overhead**. The main tools used here were Microsoft Excel or Word. We can infer that this practice is **not recommendable** in comparison with the other ones.
- **[Ineffective]** Using a **bug system** for tracking TD is not considered as contributing to a better level of awareness or tracking compared to the other techniques, and it has a slightly higher overhead. We would infer that this is also **not the best way of tracking TD**.
- **[Ineffective] Test coverage does not** seem to **contribute** too much **to the awareness and tracking level**, although it does not involve much overhead. This might be because test coverage is related to only a small part of TD.

References and Resources

1. Ward Cunningham. 1992. The WyCash portfolio management system. In Addendum to the proceedings on Object-oriented programming systems, languages, and applications (Addendum) (OOPSLA '92), Jerry L. Archibald and Mark C. Wilkes (Eds.). ACM, New York, NY, USA, 29-30. DOI=http://dx.doi.org/10.1145/157709.157715
2. S. McConnell, "Managing technical debt (slides)," in Workshop on Managing Technical Debt (part of ICSE 2013): IEEE, 2013
   - http://2013.icse-conferences.org/documents/publicity/MTD-WS-McConnell-slides.pdf
3. Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. 2012. Technical Debt: From Metaphor to Theory and Practice. IEEE Softw. 29, 6 (November 2012), 18-21. DOI: https://doi.org/10.1109/MS.2012.167
4. Chris Sterling, 2010. Managing Software Debt: Building for Inevitable Change. Addison-Wesley Professional; 1 edition, ISBN-10: 0321948610. [a good resource to know technical debts]
5. Lim, N. Taksande, and C. Seaman, “A balancing act: What software practitioners have to say about technical debt,” IEEE Software. DOI: 10.1109/MS.2012.130
6. Jesper Boeg, Successfully balancing technical debt and new features – staying in the “flow-zone” or how to get back in there. http://agileupgrade.com/successfully-balancing-technical-depth-and-new-features-staying-in-the-flow-zone-or-how-to-get-back-in-there/
7. Antonio Martini, Tersese Besker, Jan Bosch: Technical Debt tracking: Current state of practice: A survey and multiple case study in 15 large organizations. Sci. Comput. Program. 163: 42-61 (2018)
8. Terese Besker, Antonio Martini, and Jan Bosch. 2019. Technical debt triage in backlog management. In Proceedings of the Second International Conference on Technical Debt (TechDebt '19). IEEE Press, Piscataway, NJ, USA, 13-22. DOI: https://doi.org/10.1109/TechDebt.2019.00010